Review package: operation-scoped cancellation (B1–B7)

> ***Scope note.*** *This packet reviews the* ***operation-scoped-cancellation*** *work. It supersedes the earlier "browser lifetime + stop rule S4" packet (candidate* `9a1b931`*, base* `b676c2d`*), which is a separate change preserved in git history. Nothing here touches BM25/aboutness/discovery/counters/scoring/operating-settings,* `BATCH_STOP_RULE_FAILED_JOBS` *stays unset, and there is no quarantine-recovery override.* ***This is a local feature-branch review candidate only: no merge, no deploy, no paid/Railway runs, no live web/LLM calls.***

Branch	`fix/operation-scoped-cancellation`
Base (production)	**`origin/main = 8c03b07449701a7fb83dc062abbfe6726cf7c208`** — untouched; nothing pushed to it
Release candidate	**`45cc661d2898e52ad8512083929535d1667050d7`**
Lineage	`f764c95` (B1 recovery scope + B6 killable pdf-parse subprocess) → `b317833` (B2/B3/B7 production-boundary tests + B7 test seam) → `0d05556` (recovery window = real remaining fetch-phase budget) → `782f99d` (bounded in-flight browser interruption) → `e4894ae` (obsolete-work settlement for coalesced callers) → **`45cc661`** (real-IPC proc-admission withdrawal test + explicit unattempted-URL outcomes)
Gates	No new env switch. The cancellation machinery is active on the isolated-child path that production already runs (`WORKER_ISOLATION_ENABLED=true`). `BATCH_STOP_RULE_FAILED_JOBS` is **not** part of this change and stays unset.
Delivery constraints honoured	Feature branch only; **no push to origin/main**; no merge; no deploy; no paid/Railway runs; no real web/LLM calls.

1. What changed (`origin/main..45cc661`)

Code:

- **`server/lib/pipeline.ts`** —

- `runCompanyPdfRecovery` arms **one** `OperationScope` per company PDF recovery, caps its deadline by `min(PDF_RECOVERY_MAX_BUDGET_MS, remainingMs)`, propagates that scope to **every** fetch call (`fetchFn(origin, slice, { outcomes, scope })`), stops dispatching once the scope aborts (leaving the untried tail retryable), and caps total attempts across origins.
- **(`0d05556`)** `remainingFetchBudgetMs(fetchPhaseStart)` returns the real remaining fetch-phase budget (configured budget minus elapsed, clamped to `RECOVERY_WINDOW_FLOOR_MS`), and the recovery call site now feeds **that** as `remainingMs` — so the production recovery window is the per-company remaining budget, not the raw 600000 ms constant. The constant itself is unchanged.
- **(`45cc661`)** `markUnattemptedCancelled(companyId, companyName)` + const `NOT_ATTEMPTED_CANCELLED_REASON = "not_attempted_cancelled"`: on a cancel break in the fetch loop, every still-`pending` accepted document is recorded fetch-dead with that reason, so cancellation no longer leaves URLs silently `pending`. Wired into the top-of-loop cancel break (non-fatal, wrapped in try/catch).
- **`server/lib/processor.ts`** —
- **B2:** `acquireProcSlot` queued waiter is cancellable — on abort it withdraws from the local broker queue and consumes no slot; grant/abort races hand the slot straight back. In the isolated-child path it withdraws across IPC via `remoteAcquire("proc", signal, () => new OperationAbortedError(…, "proc_admission"))`.
- **B3:** `runProcessCollect` registers an abort listener that SIGKILLs a hung extraction child at the abort instant.
- **B6:** pdf-parse runs in a **killable subprocess** (`extractViaPdfParseSubprocess` → `pdf-parse-worker.cjs` via `runProcessCollect`), not in-process — a genuine execution boundary.
- **B7:** `processDocument` coalesces concurrent same-URL callers to one fetch **only while the in-flight scope is live**; a caller arriving after that scope aborts starts its own attempt (liveness guard). **(`e4894ae`)** a caller that *coalesced onto* an in-flight op and whose **own** scope then aborts self-settles via `Promise.race([existing.promise, callerScope.whenAborted("coalesced_wait")])` instead of hanging on the obsolete shared op.
- **(`782f99d`)** `installBoundedPageInterrupt` + `BROWSER_ABORT_GRACE_MS` (500 ms): on scope abort, an in-flight browser op on the **`fetchPdfViaBrowser`** path is driven to settle within the grace bound + a gate round-trip instead of running to its own timeout. **Wired only on the PDF path; the `fetchWithBrowser` HTML path is not wired** (see §6.1).

Tests added / present (all under `server/lib/`): `company-pdf-recovery-boundary.test.ts` (B1, incl. B1-7 recovery window), `cancellable-proc-admission-boundary.test.ts` (B2 in-process), `isolated-proc-admission-boundary.test.ts` (B2 across real fork+IPC) + fixture `__fixtures__/proc-admission-child.ts`, `stalled-extraction-boundary.test.ts` (B3), `pdf-parse-subprocess-equivalence.test.ts` (B6), `inflight-coalescing-boundary.test.ts` (B7, incl. B7-3 obsolete-work settlement), `browser-inflight-interrupt.test.ts` (bounded in-flight page interrupt, real Chromium), `unattempted-cancelled-boundary.test.ts` (explicit unattempted-URL outcomes, real Postgres), `operation-scoped-cancellation.test.ts` (scope primitives A/B/C) and the pre-existing `browser-close-order.test.ts` (real-Chromium close/reap ordering).

---

2. Coverage matrix

Full requirement→test→assertion map, with the realism level of each case and the honest gaps, is in **COVERAGE_MATRIX.md**. Headline: **11 test files, all green on `45cc661`** — 33 `node:test` subtests across 10 files + `scoring-cancellation.unit` (60 passed). Realism spans a real-killed child process (B3), a real fork+IPC admission withdrawal (B2b), a real Chromium child close/reap on four exit paths,

a real Chromium page bounded-interrupt, real Postgres terminal-state assertions, and the real production broker/scope functions in-process for the rest.

---

3. Mutation-fail demo per guard (all run this session, applied → observed → reverted clean)

Each guard is shown to be load-bearing: remove it and the named assertion fails. Every mutation was reverted and the tree re-verified clean (`grep -rn MUTATION_MARKER server/` → none; `git diff --quiet HEAD -- server/lib/processor.ts server/lib/pipeline.ts` → clean).

Gap	Mutation	Result	Evidence
B1 scope propagation	`fetchFn(origin, slice, { outcomes, scope })` → `{ outcomes }`	**B1-1 fails**	`AssertionError … a scope was passed to the fetch boundary`
B1-7 recovery window	`remainingFetchBudgetMs` body → `return budgetMs;` (`MUTATION_MARKER_B2`)	**B1-7 fails**	`6 pass / 1 fail` — window no longer shrinks with elapsed
B2 queued-waiter abort	remove `sig.addEventListener("abort", onAbort…)`	**B2-1 fails**	`'hung' !== 'settled'` ; B2-2/3/4 cascade `proc slot leaked`
B2b IPC withdrawal error type	`() => new OperationAbortedError(…, "proc_admission")` → `() => new Error("MUTATION_MARKER_B5 …")`	**isolated-proc-admission fails fast**	child withdraws at ~700 ms but `isOperationAborted: false`
B3 stalled-child kill	disable abort registration	**B3 fails**	`parent settled promptly after abort (elapsed=60005ms)` — child ran the **full 60 s**
B7 coalescing	`if (false && existing …)`	**B7-1 fails**	`2 !== 1` (two fetches)
B7 liveness	ignore `scope.aborted()`	**B7-2 fails**	`1 !== 2` (new caller coalesced onto aborted entry)
B7-3 coalesced self-settle	`Promise.race([existing.promise, callerScope.whenAborted(…)])` → `return existing.promise;` (`MUTATION_MARKER_B4`)	**B7-3 fails**	`2 pass / 1 fail` — coalesced caller hangs past its own abort
B-browser bounded interrupt	`installBoundedPageInterrupt` `on-`	**browser-inflight-interrupt fails**	page settled **8169 ms** vs `< 3000 ms` bound

Gap	Mutation	Result	Evidence
	`Abort` body → no-op (`MUTATION_MARKER_B3`)		
B6(unattempted) terminal outcomes	pending filter `&& false` (`MUTATION_MARKER_B6`)	**B6-1 fails**	`0 !== 3` — pending docs silently dropped again

B7 needs three mutants because the coalesce / liveness / self-settlement clauses are independently load-bearing. The B2b *error-type* mutation is the load-bearing one for the IPC path; a weaker `cancelAcquire`-suppression mutation did **not** fail (child-exit cleanup masks it) and that masking is documented in COVERAGE_MATRIX §2b rather than hidden.

---

4. Gate results (`45cc661` , this session)

All 11 cancellation test files green (fixtures: local `ciq-int-pg` on 55432, `ciq-int-redis` on 56379; no production `DATABASE_URL` ; `PDF_RECOVERY_MAX_BUDGET_MS=600000`):

```
operation-scoped-cancellation        pass 7  fail 0
company-pdf-recovery-boundary        pass 7  fail 0   (B1-7 added)
cancellable-proc-admission-boundary  pass 4  fail 0
isolated-proc-admission-boundary     pass 1  fail 0   (real fork+IPC)
inflight-coalescing-boundary         pass 3  fail 0   (B7-3 added)
stalled-extraction-boundary          pass 2  fail 0
pdf-parse-subprocess-equivalence     pass 2  fail 0
browser-close-order (real Chromium)  pass 4  fail 0
browser-inflight-interrupt (real)    pass 1  fail 0
unattempted-cancelled-boundary (PG)  pass 2  fail 0
scoring-cancellation.unit            60 passed, 0 failed   (custom runner)
```

33 `node:test` subtests + 60 scoring-unit assertions, **0 failures**. `browser-close-order` and `browser-inflight-interrupt` ran against a **real** Chromium (`PUPPETEER_EXECUTABLE_PATH`), not skipped; `isolated-proc-admission-boundary` spawned a **real** OS child over the production IPC channel.

---

5. Normalized tsc comparison

`npx tsc --noEmit -p tsconfig.json` , error sets path/line-normalized and de-duplicated, candidate vs base (base run in a worktree checked out at `origin/main` = `8c03b07` with the same `node_modules`). Normalization: `grep 'error TS' | sed -E 's/\(([0-9]+),([0-9]+)\)//' | sort -u` (unique error *signatures*, not raw line count).

	unique error signatures
base `8c03b07`	13
candidate `45cc661`	12
new errors in candidate (`comm -13`)	**0**
errors removed by candidate (`comm -23`)	1 — `server/lib/processor.ts … TS7016: Could not find a declaration file for module 'pdf-parse'` (eliminated because B6 moved pdf-parse out of the in-process import)

The candidate introduces **zero** new type errors and removes one pre-existing `pdf-parse` declaration warning. All 12 remaining signatures are pre-existing vitest/`any` noise identical to base. (This counts unique error *signatures*; the earlier packet's "62/61" figure counted raw error *lines* — the invariant is the same either way: **0 new errors**.)

---

6. Remaining unbounded / unproven paths (honest boundary)

1. **Active in-page CDP interruption is bounded on one path only.** `installBoundedPageInterrupt` bounds an in-flight CDP op to `BROWSER_ABORT_GRACE_MS` + a gate round-trip on the **`fetchPdfViaBrowser`** path (proven in `browser-inflight-interrupt.test.ts` with a real Chromium page, §5b of the matrix). The **`fetchWithBrowser` HTML path is still not wired** to this helper — a fetch/navigation/`page.evaluate`/WAF challenge already executing on that path when the scope aborts is still not torn down mid-operation; it falls back to the next admission gate (B2/B7 refusal), dispatch suppression (B1-3), and ultimately the job-isolation SIGKILL (browser-close-order child-death path). This is a real, named remaining gap.
2. **Deadline is bounded around the browser fetch, not threaded into it.** B1 proves the scope is passed and that dispatch stops after the deadline; it does not apply a per-request socket timeout equal to `remainingMs` inside `fetchIssuerPdfsWithPrimedSession`. The effective ceiling on an already-started browser fetch on the un-wired path is the pipeline deadline → SIGKILL.
3. **(RESOLVED in this candidate)** The production recovery window previously degenerated to the `PDF_RECOVERY_MAX_BUDGET_MS` (600000) constant because the call site omitted `remainingMs`. It now feeds `remainingFetchBudgetMs(fetchPhaseStart)` — the configured budget minus elapsed, clamped to a floor — so the per-company remaining budget is honoured in production, not just in the test (B1-7 + mutation). The constant is unchanged; only the derivation changed.
4. **Specific production stalls unobserved.** The Idemitsu ~18-minute single step and the AXA analysis-prep ~7m26s stall (`TIMEOUT_INVESTIGATION.md`) are not reproduced here and are **not** claimed fixed by this patch.

---

7. Claims kept narrow

- **Candidate only.** `45cc661` is a review candidate on a feature branch. Nothing was merged, deployed, or run on Railway/paid providers. `origin/main` is untouched at `8c03b07` (verified: `git rev-parse origin/main` = `8c03b07…`; local `main` = `8c03b07…`).
- **No full scoring fixture.** Per the reviewer's clarification, no end-to-end scoring run is included; the behavioural tests drive the real caller-to-recovery boundary and the real admission/coalescing/subprocess/IPC/Postgres functions directly.
- **Mutations are session-local and reverted.** Every §3 mutation was applied, observed to fail the named assertion, and reverted in this session; `grep -rn MUTATION_MARKER server/` is empty and `git diff --quiet HEAD -- server/lib/processor.ts server/lib/pipeline.ts` is clean.
- **What is proven** is the requirement→assertion map in COVERAGE_MATRIX.md plus the §3 mutation demos. **What is not proven** is in §6 and COVERAGE_MATRIX §6 — read the green suite as exactly that boundary, no wider.

---

8. Reviewer's 6-point clarification — status

1. **Behavioural recovery boundary (not callee+grep).** Done — `runCompanyPdfRecovery` is real production code called in the production pipeline; B1-1..B1-7 assert scope construction, deadline capping, no-dispatch-after-expiry, retryable tail, healthy path, expiry-through-the-real-admission-boundary, **and** that the production window is the real remaining budget (B1-7). Mutations: scope-drop → B1-1 fail; `return budgetMs` → B1-7 fail (§3).
2. **Chromium realism + coverage matrix.** Done — COVERAGE_MATRIX.md states per case whether it is real-Chromium-child (R), real-Chromium-page (R(page)), real-subprocess (S), real fork+IPC (I), real-Postgres (D), or broker/unit (U), and lists the active-interruption gap explicitly (including the still-un-wired `fetchWithBrowser` HTML path).
3. **Fixture infra.** Done — `ciq-int-pg` (55432) / `ciq-int-redis` (56379), no production `DATABASE_URL`.
4. **B6 killable execution boundary.** Done — subprocess, not in-process size cap (§1); `pdf-parse-subprocess-equivalence.test.ts` green. Admission withdrawal additionally proven across a real fork+IPC boundary (`isolated-proc-admission-boundary.test.ts`).
5. **Recovery budget value retained, semantics documented.** `PDF_RECOVERY_MAX_BUDGET_MS=600000` retained; the production window now derives from the real remaining fetch-phase budget (B1-7), resolving the earlier "degenerates to the constant" gap. Documented in §6.3 and tested.
6. **Delivery: feature branch, no origin/main, no deploy/paid runs.** Honoured (§ header + §7).